

spaCy, scispaCy, and MedSpaCy

Lauren Escobedo
AI in Health AI 380 H
Professor Ying Ding

Data Preprocessing

1. Investigate ICD-9 diagnosis cohorts. Looking for:
 - a. Reasonable amount of data to work with: $2,000 < \text{num_discharges} < 3,000$
 - b. *This rule was derived after many hours of iteration after waiting for my laptop to process large chunks of data. I eventually settled on this range, and later pivoted to work with even smaller samples of the notes due to CPU constraints.*
2. Selected hyperpotassemia - 2,169 discharges.
3. Download discharge notes for this cohort.
 - a. I had to add a limit of 100,000 rows to the query for feasibility on my machine
4. Pickle the notes to a file for usage across all 3 tools

spaCy

spaCy

1. Create a corpus of entities, as shown in the example notebook. I also created an entity dictionary for use as the t-SNE plot legend later.
2. I used a reproducible sample of 20% of the total notes.
3. Pickle the resulting DataFrames.
4. Code and t-SNE plot on following slides.

spaCy NER pipeline

```
PE = "spacy_entities.pkl"
PC = "spacy_corpus_entities.pkl"
SAMPLE_FRAC = 0.2
MAX_CHARS = 20000

if os.path.exists(PE) and os.path.exists(PC):
    spacy_entities = pickle.load(open(PE, "rb"))
    spacy_corpus = pickle.load(open(PC, "rb"))
else:
    nlp = spacy.load("en_core_web_sm", exclude=["parser", "lemmatizer", "attribute_ruler", "tagger"]) # Do not load the parser, lemmatizer, attribute ruler, and tagger

    text = noteevents_df['text'].dropna().sample(frac=SAMPLE_FRAC, random_state=42)
    text = text.str.slice(0, MAX_CHARS)

    spacy_entities = {}
    spacy_corpus = []

    for doc in tqdm(nlp.pipe(text, batch_size=32, n_process=4), total=len(text), desc="Extracting entities"):
        ents = [e.text.lower() for e in doc.ents]
        spacy_corpus.append(ents)

        for entity in doc.ents:
            spacy_entities.setdefault(entity.text.lower(), entity.label_)

    pickle.dump(spacy_entities, open(PE, "wb"))
    pickle.dump(spacy_corpus, open(PC, "wb"))
```

word2vec on spaCy entities

```
"""
Train word2vec on the corpus
"""

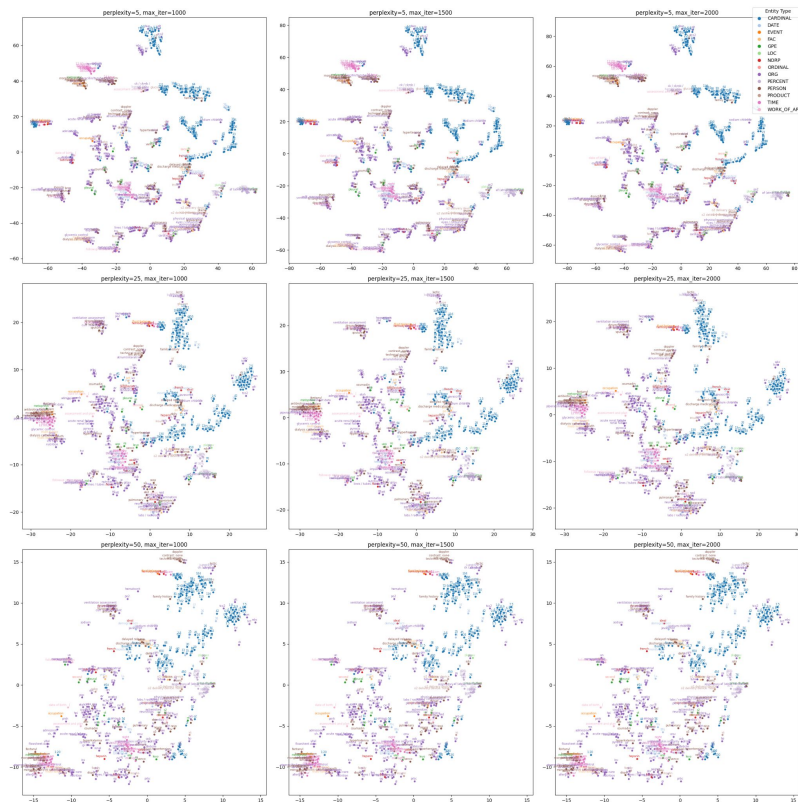
from gensim.models import Word2Vec

# Removing these entities because it filled the plot with numbers
# skip = {"CARDINAL", "PERCENT", "QUANTITY", "TIME", "DATE", "ORDINAL", "MONEY"}
skip = []

corpus_filtered = [
    [e for e in note if spacy_entities.get(e, "") not in skip]
    for note in spacy_corpus
]

model = Word2Vec(corpus_filtered, min_count=5)
```

Tuning spaCy t-SNE plots

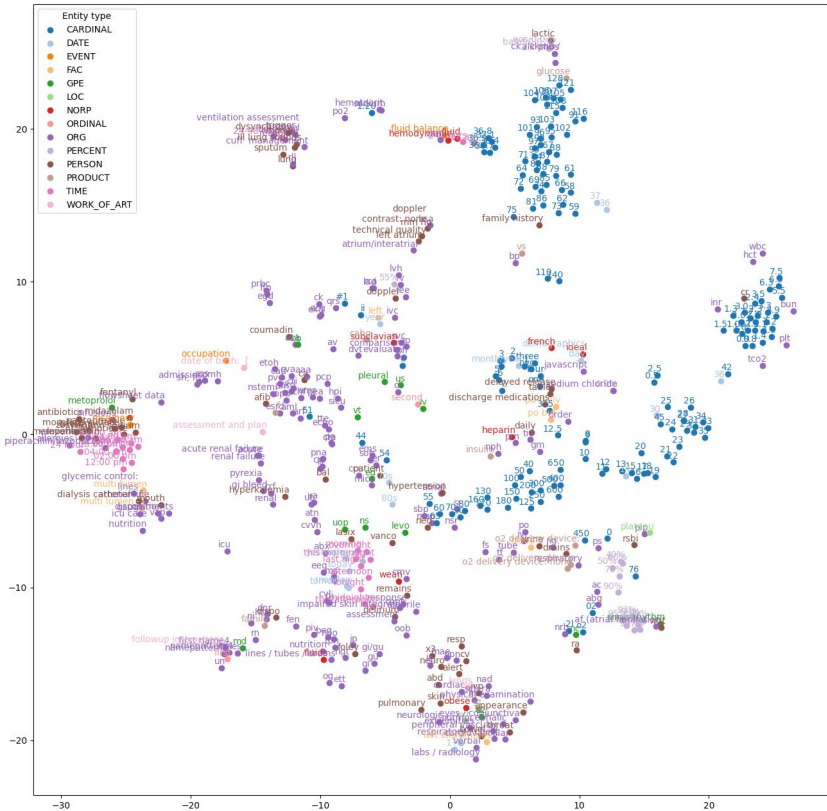


The best tuned plot appears to me to be the plots in the position [1,0:2].

This uses perplexity=25 and max_iter=1000-2000.

Since the plot seems to converge at 1000, there is no need to require more iterations.

Tuned spaCy t-SNE plot



We can see clear groupings surrounding entities, although it is clear that many entities are miscategorized.

scispaCy

scispaCy

1. Again, create a corpus of entities, as shown in the example notebook. I also created an entity dictionary for use as the t-SNE plot legend later.
2. I used a reproducible sample of 20% of the total notes.
3. Pickle the resulting DataFrames.
4. Code and t-SNE plot on following slides.

scispaCy NER pipeline

```
PE = "scispaCy_entities.pkl"
PC = "scispaCy_corpus_entities.pkl"
SAMPLE_FRAC = 0.2
MAX_CHARS = 20000

if os.path.exists(PE) and os.path.exists(PC):
    scispaCy_entities = pickle.load(open(PE, "rb"))
    scispaCy_corpus = pickle.load(open(PC, "rb"))
else:
    nlp = spacy.load("en_ner_bc5cdr_md", exclude=["parser", "lemmatizer", "attribute_ruler", "tagger"])

    text = noteevents_df['text'].dropna().sample(frac=SAMPLE_FRAC, random_state=42)
    text = text.str.slice(0, MAX_CHARS)

    scispaCy_entities = {}
    scispaCy_corpus = []

    for doc in tqdm(nlp.pipe(text, batch_size=32, n_process=4), total=len(text), desc="Extracting entities"):
        ents = [e.text.lower() for e in doc.ents]
        scispaCy_corpus.append(ents)

        for entity in doc.ents:
            scispaCy_entities.setdefault(entity.text.lower(), entity.label_)

    pickle.dump(scispaCy_entities, open(PE, "wb"))
    pickle.dump(scispaCy_corpus, open(PC, "wb"))
```

word2vec on scispaCy entities

```
"""
Train word2vec on the corpus
"""

from gensim.models import Word2Vec

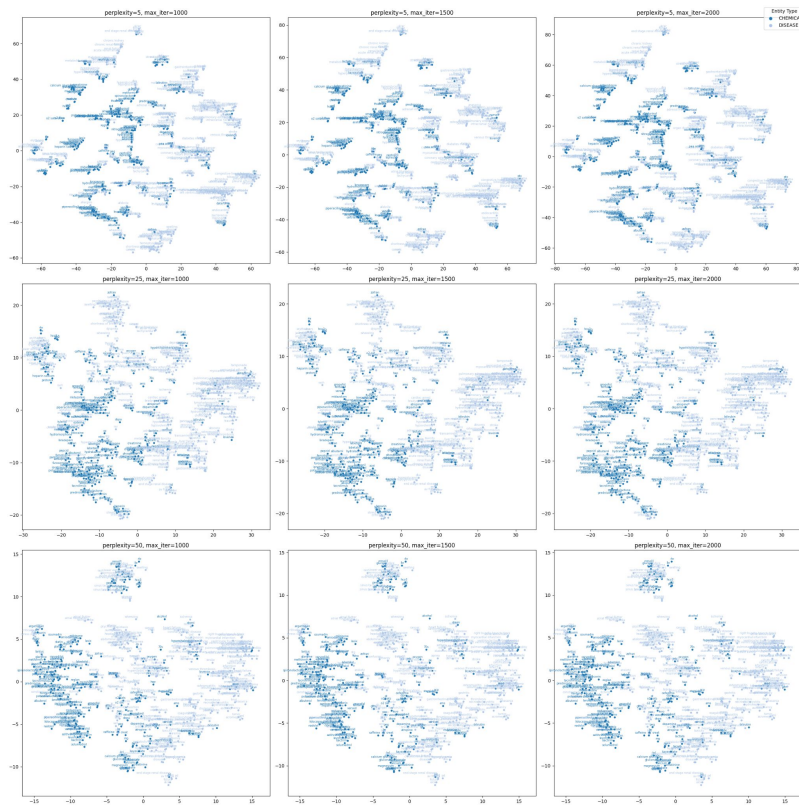
# Removing these entities because it filled the plot with numbers
# skip = {"CARDINAL", "PERCENT", "QUANTITY", "TIME", "DATE", "ORDINAL", "MONEY"}
skip = []

corpus_filtered = [
    [e for e in note if scispaCy_entities.get(e, "") not in skip]
    for note in scispaCy_corpus
]

model = Word2Vec(corpus_filtered, min_count=5)
```

✓ 2.2s

Tuning scispaCy t-SNE plots

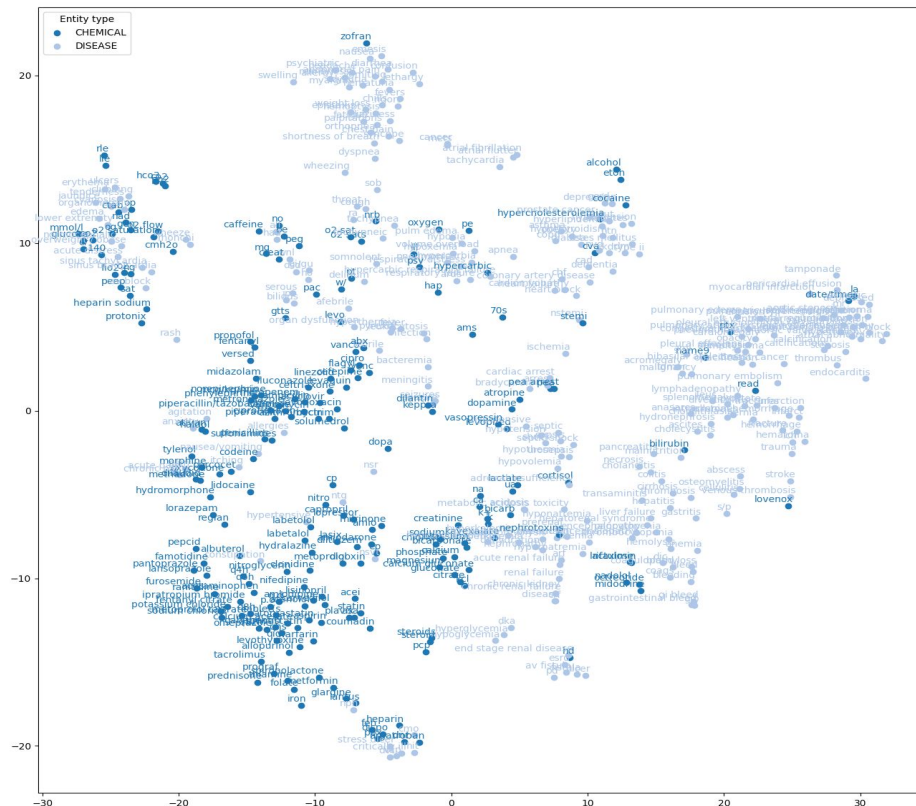


Again, the best tuned plot appears to me to be the plots in the position [1,0:2].

This uses perplexity=25 and max_iter=1000-2000.

Since the plot seems to converge at 1000, there is no need to require more iterations.

Tuned scispaCy t-SNE plot



We can see clear groupings surrounding entities, and it's clear that the entities are correctly categorized this time.

MedSpaCy

MedSpaCy

1. Again, create a corpus of entities, as shown in the example notebook. I also created an entity dictionary for use as the t-SNE plot legend later.
2. I used a reproducible sample of **5%** of the total notes.
 - a. **I had to reduce the sample size, this method was far too computationally expensive for my laptop. I understand this has effects on the validity of the conclusions across all three methods, but it was unavoidable in this case.**
3. Pickle the resulting DataFrames.
4. Code and t-SNE plot on following slides.

MedSpaCy NER pipeline

```
PE = "scispacy_entities.pkl"
PC = "scispacy_corpus_entities.pkl"
SAMPLE_FRAC = 0.2
MAX_CHARS = 20000

if os.path.exists(PE) and os.path.exists(PC):
    ... scispacy_entities = pickle.load(open(PE, "rb"))
    ... scispacy_corpus = pickle.load(open(PC, "rb"))
else:
    ... nlp = spacy.load("en_ner_bc5cdr_md", exclude=["parser", "lemmatizer", "attribute_ruler", "tagger"])

    ... text = noteevents_df['text'].dropna().sample(frac=SAMPLE_FRAC, random_state=42)
    ... text = text.str.slice(0, MAX_CHARS)

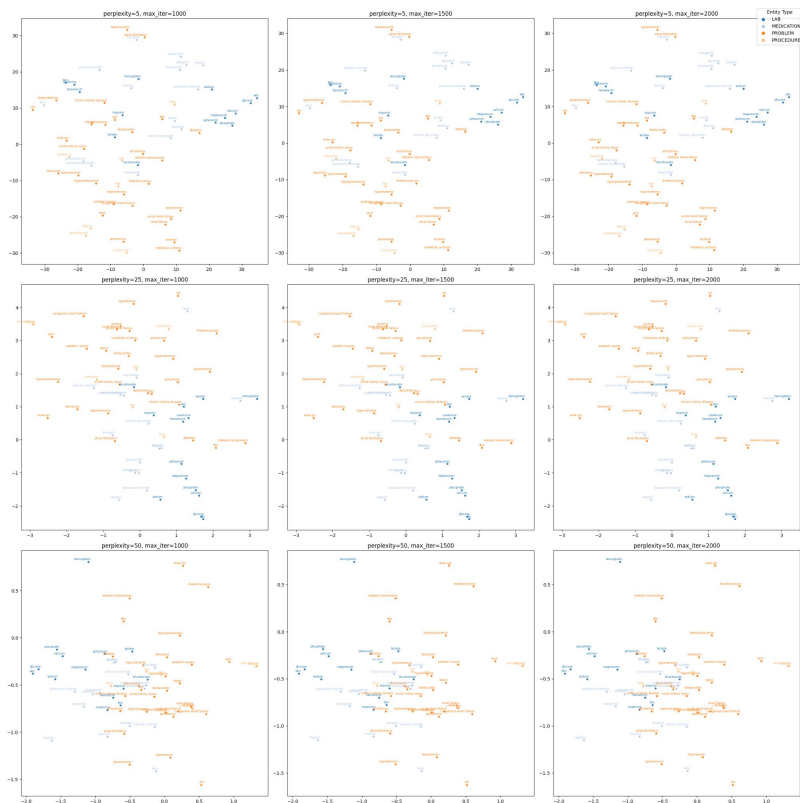
    ... scispacy_entities = {}
    ... scispacy_corpus = []

    ... for doc in tqdm(nlp.pipe(text, batch_size=32, n_process=4), total=len(text), desc="Extracting entities"):
    ...     ... ents = [e.text.lower() for e in doc.ents]
    ...     ... scispacy_corpus.append(ents)

    ...     ... for entity in doc.ents:
    ...     ...     ... scispacy_entities.setdefault(entity.text.lower(), entity.label_)

    ... pickle.dump(scispacy_entities, open(PE, "wb"))
    ... pickle.dump(scispacy_corpus, open(PC, "wb"))
```

Tuning MedSpaCy t-SNE plots

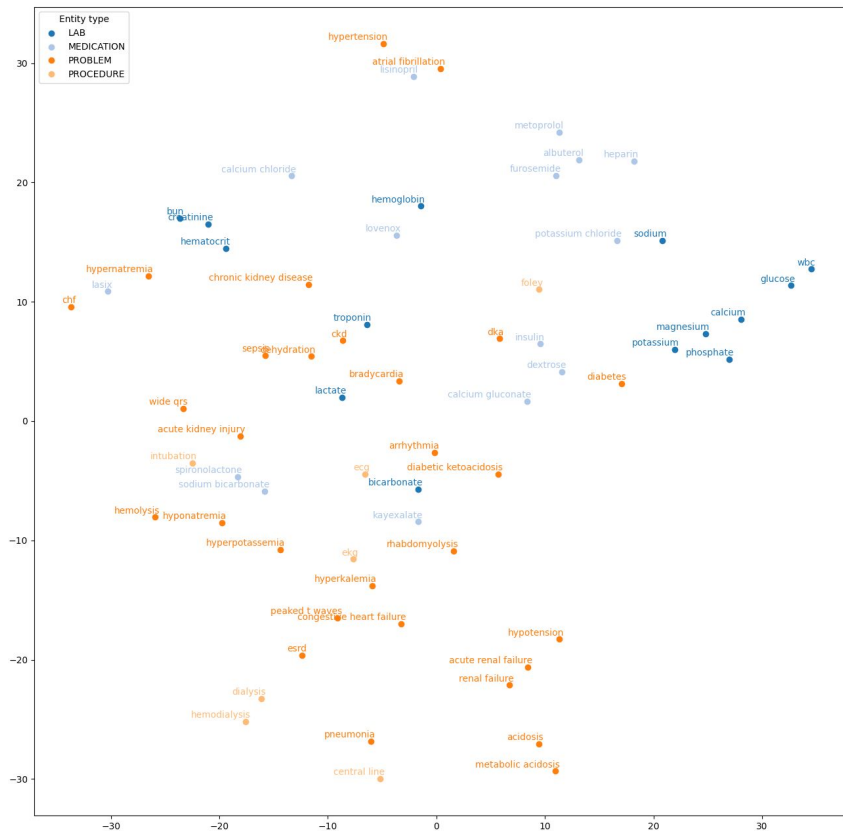


In this case, the perplexity 5 plots look best to me. They accurately group labs and problems the best.

This uses perplexity=5 and max_iter=1000-2000.

Since the plot seems to converge at 1000, there is no need to require more iterations.

Tuned scispaCy t-SNE plot



We can see the labs grouped well on the right, as well as related problems grouped near the bottom.

Medications are scattered, but related medications are in the upper right quadrant.